

Lab 3: Architectural Pattern Justification

Student Project Portfolio & Showcase App

We chose Layered Architecture for the Student Project Portfolio & Showcase App.

Architectural Choice

Layered architecture with a Presentation layer (Web UI), a Business layer (Project Manager, Grading Service, Leaderboard Service, Voting Service), an Integration layer (GitHub Adapter, Identity Adapter) and a Data layer (Database). Components talk only through the provided and required interfaces shown in the component diagram.

Two Reasons

- **The use cases map cleanly onto layers.** Profile creation and submission (UC-01, UC-03), grading (UC-04), voting (UC-05, UC-07) and the leaderboard (UC-08) are each one business responsibility with a UI on top and shared data underneath. The app is a single, moderately sized system for a college showcase, so Microservices would add deployment, networking and data-consistency overhead we do not need, and plain Client-Server would leave the business rules unstructured.
- **External systems are isolated behind replaceable components.** GitHub link validation (UC-02) and voter eligibility (UC-06) sit in adapters exposing IRepoValidation and IEligibility. If GitHub's API or the Identity Service changes, only that adapter changes; the Project Manager and Voting Service are untouched. This is the replaceable, black-box behaviour components are meant to provide.

Security Advantage

Only the Web UI is exposed to users. The Database is reachable solely through IDataAccess from the business layer, so no client can query or modify votes or grades directly. Vote eligibility (UC-06) and suspected-vote flagging (UC-07) run server-side in the Voting Service, and grading is restricted to judges in the Grading Service, so these checks cannot be bypassed from the browser. Each layer boundary is also a single place to enforce authentication and input validation.

Performance Benefit

Because all components run in one deployment, calls between layers are in-process rather than network hops, avoiding the latency of a distributed design. The leaderboard is the most read-heavy feature (UC-08); the Leaderboard Service is its own component, so it can cache computed rankings and refresh them only when a new grade or vote arrives, instead of recomputing on every page view. During peak voting, the whole application can also be scaled horizontally behind a load balancer.

Trade-off Acknowledged

Layered systems scale as one unit and can develop tight coupling between layers. For this project's size and expected load, that cost is smaller than the operational complexity of Microservices.